

Colab 連結

<https://colab.research.google.com/drive/1ygozOCnm4WtPhehhHIY85WFTbl5bsxg2?usp=sharing>

System Prompts or Agent Instructions

```
system_writer = """
You are a Python Code Generator.
Based *only* on the user's request, write Python code that fulfills the requirement.
Output *only* the complete Python code, *NOT* enclosed in triple backticks (``python ... ``)*.
Do not add any other text before or after the code block.
"""

system_reviewer = """
You are an expert Python Code Reviewer.
Your task is to provide constructive feedback on the provided code.

    **Code to Review:**
    ```python
 {first_version}
    ```

    **Review Criteria:**
    1.  **Correctness:** Does the code work as intended? Are there logic errors?
    2.  **Readability:** Is the code clear and easy to understand? Follows PEP 8 style guidelines?
    3.  **Efficiency:** Is the code reasonably efficient? Any obvious performance bottlenecks?
    4.  **Edge Cases:** Does the code handle potential edge cases or invalid inputs gracefully?
    5.  **Best Practices:** Does the code follow common Python best practices?

    **Output:**
    Provide your feedback as a concise, bulleted list. Focus on the most important points for improvement.
    If the code is excellent and requires no changes, simply state: "No major issues found."
    Output *only* the review comments or the "No major issues" statement.
    """
```

CoT 改寫版本的兩階段與 Gradio 的對話結果

程式優化小幫手 (Reflection Agent)

請輸入你今天寫的程式

計算Legendre方程式從0加到n(使用者輸入的結果)

生成程式碼 & 修正建議

✪ 第一版程式碼 (model_writer)

```
def legendre_sum(n, x):
    """Calculate the sum of Legendre polynomials P_0(x) to P_n(x)"""
    if n < 0:
        return 0.0
    # P_0(x) = 1, P_1(x) = x
    p0 = 1.0
    if n == 0:
        return p0
    p1 = x
    total = p0 + p1
    for k in range(1, n):
        # recurrence: (k+1) P_{k+1}(x) = (2k+1) x P_k(x) - k P_{k-1}(x)
        pk1 = ((2 * k + 1) * x * p1 - k * p0) / (k + 1)
        total += pk1
        p0, p1 = p1, pk1
    return total
```

✪ 修改建議 (model_reviewer)

- Add type hints and a more detailed docstring (e.g., describe the parameters and return type).
- Use clearer variable names such as 'p_prev', 'p_curr', 'p_next' instead of 'p0', 'p1', 'pk1'.
- Separate the input-validation logic into small helper functions to keep 'if __name__ == "__main__"' tidy.

- **Efficiency**

- The algorithm is already optimal—O(n) time and O(1) extra space. No performance issues.

- **Edge Cases**

- Consider validating that 'x' is a finite number ('math.isfinite(x)') to avoid propagating infinities or NaNs.
- The function currently accepts any real 'x'; if the intended domain is '[-1, 1]', enforce that range and raise a clear error otherwise.

- **Best Practices**

- Use 'sys.exit(1)' instead of the built-in 'exit(1)' for script termination.

✪ 第二版程式碼 (model_writer 改寫)

```
if __name__ == "__main__":
    sys.exit("錯誤: x 必須是有限的實數。")
else:
    sys.exit("錯誤: x 必須在區間 [(domain[0], (domain[1])] 內的有限實數。")

def main() -> None:
    n = _read_non_negative_int("請輸入 n (非負整數):")
    # 若希望限制 x 的範圍, 可修改 domain; 此處以 [-1, 1] 為例
    x = _read_finite_float("請輸入 x (實數, 如 0.5):", domain=(-1.0, 1.0))
    result = legendre_sum(n, x)
    print(f"Legendre 多項式從 P_0 到 P_n 在 x={x} 處的總和為: {result}")

if __name__ == "__main__":
    main()
```

🛠️ 程式優化小幫手 (Reflection Agent)

請輸入你今天寫的程式

使用者輸入n，請計算1^2+2^2+...+n^2的結果

生成程式碼 & 修正建議

🌟 第一版程式碼 (model_writer)

```
def sum_of_squares(n: int) -> int:
    """Calculate 1^2 + 2^2 + ... + n^2 using the closed-form
    formula."""
    return n * (n + 1) * (2 * n + 1) // 6

if __name__ == "__main__":
    try:
        n_input = input().strip()
        n = int(n_input)
        if n < 0:
            raise ValueError("n must be a non-negative integer")
        print(sum_of_squares(n))
    except Exception as e:
        print(f"Error: {e}")
```

🔧 修改建議 (model_reviewer)

- The implementation of the closed-form formula is correct and uses integer division, so it works for all non-negative integers without overflow issues in Python.
- Type hinting and the docstring make the function's purpose clear; the code follows PEP8 naming and spacing conventions.
- The input handling catches any exception, which may hide unexpected bugs; it would be better to catch only `ValueError` (invalid integer conversion or negative value) and let other errors surface.`
- The error message is printed directly; consider writing to ``stderr` ( ` print(..., file=sys.stderr)` ) for a cleaner CLI experience.`
- Minor improvement: strip the input only after confirming it isn't empty, or provide a helpful prompt/message for the user.
- Overall, the code is correct, readable, and efficient; only the exception handling could be refined.

🌟 第二版程式碼 (model_writer 改寫)

```
import sys

def sum_of_squares(n: int) -> int:
    """計算 1^2 + 2^2 + ... + n^2 的閉合公式結果。"""
    return n * (n + 1) * (2 * n + 1) // 6

if __name__ == "__main__":
    try:
        raw_input = input("請輸入一個非負整數 n: ").strip()
        if not raw_input:
            raise ValueError("未輸入任何內容")
        n = int(raw_input)
        if n < 0:
            raise ValueError("n 必須是非負整數")
        print(sum_of_squares(n))
    except ValueError as e:
        print(f"Error: {e}", file=sys.stderr)
```